알고리즘 적용 기획서 #1

EnjoyTrip Algorithm EP4 | A12 정제영 · 박재윤 | 2026-10-15

■ 내용 : KMP를 이용한 관광지 이름·주소 검색 서비스

■ 적용 알고리즘 : KMP (Knuth-Morris-Pratt) 문자열 탐색

■ 알고리즘 개요

KMP는 패턴의 접두사와 접미사 정보를 prefix 배열에 저장하는 문자열 탐색 알고리즘이다. 비교가 실패하면 이미 일치한 구간을 다시 검사하지 않고 다음 비교 위치로 이동한다. 반복되는 문자열에서도 최악의 선형 탐색 시간을 보장하므로 관광지 목록의 이름·주소 재검색에 적용했다.

■ 적용 서비스 : 불러온 여행지에서 찾기

■ 적용 서비스 개발 개요

사용자는 지도 화면 왼쪽의 "목록 검색·거리 정렬"을 펼쳐 이름 또는 주소를 입력한다. 지역·유형·장소 이름으로 공공데이터를 먼저 조회한 후, 현재까지 불러온 목록만 다시 검색하는 기능이다. 전국의 전체 관광 데이터를 대상으로 새 API 검색을 하는 기능과 구분한다.

JavaScript 화면은 원본 관광지 목록의 식별자, 이름, 주소, 좌표와 검색어를 URL 인코딩 폼으로 Java 서버에 전달한다. Java는 검색어 앞뒤 공백을 제거하고 Locale.ROOT로 소문자화한다. 패턴의 prefix 배열은 요청당 한 번 생성하고 모든 장소에 재사용한다. 각 장소의 이름과 주소를 따로 탐색해 두 필드 경계에서 생기는 잘못된 일치를 방지한다.

검색 결과는 원래 입력 인덱스로 반환한다. 화면은 이 인덱스를 원본 데이터에 연결하므로 찜, 상세보기, 여행에 추가 기능에서 동일한 장소 식별자를 유지한다. 빈 검색어는 전체 목록을 반환하며, 일치하는 장소가 없으면 빈 결과 안내를 표시한다. 초기화하면 원본 순서를 복구하고 새로운 조회가 끝나면 검색 대상 목록도 갱신한다.

빠른 연속 입력이나 초기화 후 이전 응답이 도착해 화면을 덮어쓰지 않도록 요청 취소와 버전 검사를 함께 사용한다. 요청 제한 시간은 15초다. 서버는 장소 10,000개, 검색어 500자, 본문 8MiB를 상한으로 검증한다. 화면에 표시하는 처리 시간은 Java 계산 시간이며 통신·파싱·렌더링 시간은 포함하지 않는다.

검증은 한국어, 이모지, 영문 대소문자, 빈 문자열, 미일치, 반복 접두사와 필드 경계를 포함한다. 무작위 2,000회 입력은 Java contains 결과와 대조했다. 일반 짧은 문자열은 기존 JS includes가 더 빠를 수 있으므로, 적용 효과는 반복 입력에서의 개선과 최악 시간 보장으로 설명한다.

구현 위치: backend/src/enjoytrip/Algorithms.java · js/algorithm-ui.js

알고리즘 적용 기획서 #2

EnjoyTrip Algorithm EP4 | A12 정제영 · 박재윤 | 2026-10-15

■ 내용 : 병합 정렬을 이용한 기준 관광지 거리순 탐색 서비스

■ 적용 알고리즘 : 안정 병합 정렬 (Merge Sort), Haversine 거리 계산

■ 알고리즘 개요

병합 정렬은 목록을 작은 구간으로 나누고 정렬된 구간을 병합하는 분할 정복 알고리즘이다. 평균과 최악 모두 O(K log K)이며, 동일한 비교 값에서는 기존 순서를 유지할 수 있다. 거리 계산과 정렬을 분리해 가까운 관광지를 안정적으로 탐색하도록 적용했다.

■ 적용 서비스 : 기준 관광지와 가까운 여행지 찾기

■ 적용 서비스 개발 개요

사용자는 왼쪽 "목록 검색·거리 정렬"에서 거리 기준 장소를 선택하고 적용을 누른다. 이름·주소 검색과 함께 사용할 수 있으며, 검색어가 있으면 일치한 장소만 거리순으로 정렬한다. 기준 장소를 선택하지 않으면 검색만 수행하고 불필요한 거리 계산을 생략한다.

Java는 위도·경도가 유한한 값이며 각각 ±90도, ±180도 범위 안에 있는지 확인한다. 각 후보와 기준 장소 사이의 Haversine 대권거리를 한 번 계산하고 Ranked 객체에 장소, 원본 인덱스, km 값을 저장한다. 비교 함수에서 삼각함수를 반복 계산하지 않도록 거리 값을 미리 준비한다.

Ranked 배열을 절반씩 분할하고 두 정렬 구간에서 거리가 작은 항목을 선택해 보조 배열에 병합한다. 거리가 같으면 왼쪽 항목을 먼저 선택하여 기존 입력 순서를 보존한다. 보조 배열은 최초 한 번만 만들고 재귀 호출에서 재사용한다. 좌표가 없는 장소는 정렬상 무한대로 처리해 뒤로 보내고 응답에서는 null로 전달한다.

화면은 반환된 인덱스로 카드와 지도 목록을 갱신하고 카드에 소수 둘째 자리까지 거리를 표시한다. 이 값은 좌표 기준 직선거리이므로 도로 이동거리, 소요시간, 최적 방문 순서와 다르다는 안내를 제공한다. 결과를 필터링하는 동안 더 보기는 숨기며 초기화하면 원본 목록과 페이지 조회 상태를 복구한다.

검증에서는 빈 배열, 한 개 항목, 동거리, 좌표 누락과 다양한 입력 크기를 Java 안정 정렬 결과와 비교했다. 동일 지점 0km, 경도 1도 약 111.195km와 거리 대칭성도 검사했다. 기존 서비스에는 거리 정렬이 없었으므로 성능 실험의 선택 정렬은 교육용 기준선임을 명시한다.

구현 위치: backend/src/enjoytrip/Algorithms.java · js/algorithm-ui.js

알고리즘 적용 기획서 #3

EnjoyTrip Algorithm EP4 | A12 정제영 · 박재윤 | 2026-10-15

■ 내용 : 코사인 유사도를 이용한 취향 맞춤 여행지 추천 서비스

■ 적용 알고리즘 : 콘텐츠 기반 추천과 코사인 유사도 (Cosine Similarity)

■ 알고리즘 개요

콘텐츠 기반 추천은 사용자의 선호와 후보의 속성을 비교해 적합한 항목을 찾는 방식이다. 코사인 유사도는 두 벡터의 방향이 얼마나 비슷한지 계산한다. 이 서비스는 관광 유형 8개를 공통 속성으로 사용해 추천 점수와 추천 이유를 설명할 수 있도록 구성했다.

■ 적용 서비스 : 취향에 맞는 다음 여행지

■ 적용 서비스 개발 개요

사용자는 왼쪽 "취향 맞춤 추천"을 펼쳐 좋아하는 여행 유형을 복수 선택하고 추천 개수 3·5·10개 중 하나를 고른다. 로그인 없이 명시적 유형 선택만으로 추천받을 수 있다. 현재 여행에 담은 장소가 있으면 그 유형 분포도 반영한다. 다른 사용자의 기록이나 저장된 다른 여행계획은 사용하지 않는다.

후보 장소는 관광지·숙박·음식점·문화시설·축제·공연·여행코스·쇼핑·레포츠 중 해당 차원만 1인 벡터로 표현한다. 사용자 가중치는 선택한 유형에 3을 더하고, 현재 여행의 해당 유형 비율에 2를 곱해 더한다. 두 가중치는 학습된 값이 아니라 명시적 선택을 우선하기 위한 설계 값이다.

유형 t의 사용자 가중치를 p[t]라 하면 후보 점수는 p[t]를 사용자 벡터 길이로 나눈 값이다. 장소 벡터가 one-hot이므로 후보당 한 차원만 조회한다. 이미 여행에 담은 ID, 중복 후보 ID, ID 없는 항목과 알려지지 않은 유형은 제외하고, 양수 점수만 내림차순 정렬해 지정한 개수만 반환한다. 동점은 원본 입력 순서로 결정한다.

추천 카드에는 이름·주소, 취향 일치도와 "선택한 문화시설 유형" 같은 추천 이유를 표시한다. 담은 목록이 영향을 주면 같은 유형의 장소 개수도 보여준다. 상세보기·여행에 추가는 기존 기능에 연결한다. 선택 유형, 목록, 여행 구성이 바뀌면 이전 추천을 비우고 다시 추천받도록 하여 낡은 결과 사용을 막는다.

취향도 담은 장소도 없으면 임의의 인기 추천을 만들지 않고 유형 선택을 안내한다. 같은 유형끼리는 점수가 같으며 지역·가격·거리·실제 평점은 반영하지 않는다. 취향 일치도는 만족 확률이 아니다. 연산은 JavaScript에서 수행해 별도 HTTP 호출 없이 정적 배포에서도 실행되며, 추천 검사 212개로 제외 조건·점수·순서를 확인했다.

구현 위치: js/recommendation.js · js/recommendation-ui.js

부록 A. 요구사항과 검증

3개 기획서의 구현 근거 및 적용 범위

■ 요구사항 대응

번호	반영 내용	확인 자료
F401	문자열 탐색·분할 정복 정렬·유사도 추천 선정	기획서 #1~#3
F402	Java KMP 구현 및 정확성 검증	Algorithms.java / AlgorithmsTest.java
F403	Java 병합 정렬 구현 및 정확성 검증	Algorithms.java / AlgorithmsTest.java
F404	100/1,000/10,000개 시간·메모리 및 개선 분석	부록 B / benchmarks CSV
F405	지정 양식의 알고리즘별 기획서	이 PDF / algorithm-proposals.md

■ 이론 복잡도

N: 후보 수, T: 이름·주소 총 길이, M: 패턴 길이, K: 일치 수, R: 담은 장소 수, D: 유형 수(8). 요청 본문·파싱 공간은 아래 알고리즘 추가 공간과 별도다.

알고리즘	시간	추가 공간
KMP 검색	O(M+T+N)	O(M+K+Lmax)
거리순 정렬	O(K log K), 거리 계산 O(K)	배열 O(K), 스택 O(log K)
코사인 추천	O(R+D+N log N)	O(R+D+N)

■ 검증 결과

Java 정확성 검사 2,064개, 추천 검사 212개, HTTP 통합 사례 12개를 통과했다. KMP는 Java contains와 무작위 입력을 대조했고, 병합 정렬은 Java 안정 정렬과 비교했다. 추천은 취향 없는 시작, 담은 장소 제외, 중복, 점수 범위와 결과 순서를 확인했다.

지도 중심 UI에서 종로 검색·경복궁 기준 정렬·초기화·추천 이유 표시를 확인했다. npm test, Java 서버 실행 중 npm run test:api로 재현한다. 관광 API와 지도 실키는 로컬에서 검증하지 않았으며 샘플 모드를 사용했다.

■ 추천 점수와 해석

p[t] = 3 × 선택 여부 + 2 × 현재 여행의 해당 유형 비율. 후보 점수 = p[t] / sqrt(sum(p[i]²)). 화면에는 100배한 값을 표시한다. 취향 일치도는 만족 확률이나 평점이 아니다. 추천의 정확성 검증과 이론 복잡도만 제시하며 별도 실행 시간·추천 품질 실험은 수행하지 않았다.

참고: 제공된 EnjoyTrip Algorithm 4회차 명세서(F401~F405), APS Day29 관통 설명 자료, 사용자 제공 기획서 양식. 구현 상세는 저장소 docs/algorithm-plan.md 및 recommendation-plan.md에 기록했다.

부록 B. 성능 실험과 개선

측정일 2026-10-15 | Windows 11 amd64 | Java 21.0.11

Java 실험: heap 256MiB, seed 1604+N, 워밍업 5회 후 9회 중앙값. 입력 생성은 제외하고 정렬 입력 복사는 포함했다. 메모리는 ThreadMXBean의 현재 스레드 할당 byte로 최대 heap 사용량이 아니다.

■ 정렬: 선택 정렬 기준선과 병합 정렬

N	선택 ms	병합 ms	선택 / 병합 할당 byte
100	0.2374	0.0100	416 / 832
1,000	0.4724	0.1114	4,016 / 8,032
10,000	36.6064	1.0053	40,016 / 80,032

기존 앱에는 거리 정렬이 없었다. 선택 정렬은 교육용 기준선이다. 10,000개에서 약 36.41배 빨라졌으며 보조 배열만큼 공간을 더 사용한다.

■ 검색: 단순 탐색과 KMP

입력	N	단순 ms	KMP ms	JDK contains ms
일반 한국어	100	0.0291	0.0665	0.0228
일반 한국어	1,000	0.1021	0.0214	0.0417
일반 한국어	10,000	0.5018	0.2277	0.0587
반복 접두사	100	1.2718	0.1485	0.8340
반복 접두사	1,000	12.5613	1.3325	2.9498
반복 접두사	10,000	127.3563	16.8037	31.3202

KMP prefix 할당은 일반 24byte, 반복 입력 272byte다. 다른 두 탐색의 측정 구간 추가 할당은 0byte이며 입력·결과 전체 메모리가 아니다. 반복 입력 10,000개에서는 단순 탐색 대비 약 7.58배 개선됐다.

■ 실제 기존 JS와 Java HTTP 비교

N	기존 JS includes ms	Java HTTP ms
100	0.0085	14.9969
1,000	0.0458	15.1989
10,000	0.4442	37.1449

HTTP 비교는 워밍업 3회 후 9회 중앙값, 응답 JSON 파싱까지 포함하며 화면 렌더링은 제외한다. 일반 짧은 문자열은 기존 JS가 더 빠르다. Java의 최악 시간 보장과 전체 서비스 응답 시간을 구분한다.

적용 개선: prefix 요청당 1회, 검색 탈락 장소와 정렬 미요청 시 거리 계산 생략, 거리 사전 계산, 보조 배열 재사용. 실험 원본: docs/benchmarks/results.csv, existing-comparison.csv, environment.txt.